

Documentation: LLC Resonant Converter Simulation-Based Analysis Framework

Maikel Fernando Menke, Eduardo Bayona Blanco, Pedro Pappis, Joshua R. Neusser, Guirguis Abdelmessih, Marco Antonio Dalla Costa, Jose Marcos Alonso

I. INTRODUCTION

This document provides a detailed description of the simulation-based analysis framework developed for LLC resonant converters. The framework is implemented in Python and leverages ngspice simulations to accurately model the converter's behavior. The primary goal is to facilitate the design process by enabling rapid evaluation of different circuit configurations and operating points.

II. GENERAL FRAMEWORK OVERVIEW

Fig. 1 shows the directory tree of the proposed simulation-based analysis and optimal design framework. However, in this document, we focus on the simulation-based analysis, as the optimal design framework is discussed in detail in the main paper.

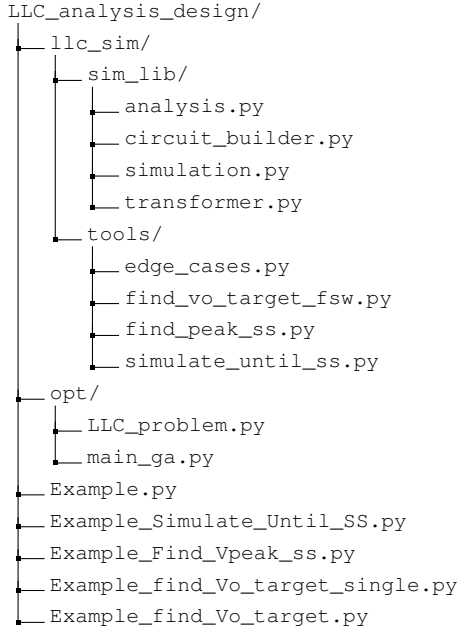


Fig. 1. Directory structure of the proposed framework.

The simulation framework is implemented in the `llc_sim` module, organized into two submodules: `sim_lib`, which provides the core circuit and simulation primitives, and `tools`, which builds higher-level algorithms to perform analysis of the LLC converter.

The `sim_lib` submodule comprises four files:

- `transformer.py` — defines the center-tap transformer as a SPICE subcircuit.

- `circuit_builder.py` — assembles the complete LLC netlist from the circuit parameters (`params`) by instantiating all components: resonant tank, transformer subcircuit, output rectifier diodes, and load.
- `simulation.py` — configures and executes the transient simulation via PySpice/ngspice, sets solver tolerances, integration method, and runs the simulation for a given time and time-step.
- `analysis.py` — post-processes the raw simulation results. Extracts waveforms and computes scalar performance metrics over the last pre-defined number of switching cycles.

The `tools` submodule comprises four files:

- `simulate_until_ss.py` — implements the function `simulate_until_steady_state`, which advances the transient simulation in successive blocks of N_{cyc} switching cycles and monitors three convergence criteria on the output voltage until steady state is declared or the limit $N_{cyc,max}$ is reached.
- `find_vo_target_fsw.py` — implements the function `find_vo_target`, which uses Brent's root-finding method to determine the switching frequency f_{sw} that produces the target output voltage V_{target} for a given operating point.
- `find_peak_ss.py` — implements the function `find_maximum_output_voltage_ss`, which locates the frequency f_{sw} that produces the maximum output voltage of the LLC converter using Brent's bounded scalar minimization.
- `edge_cases.py` — orchestrates parallel multi-operating-point validation by calling `find_vo_target` for each operating point. Results can be consolidated into a comparison table reporting key electrical quantities.

The `opt` module implements the simulation-based optimal design framework. It comprises two files:

- `LLC_problem.py` — defines the `LLC_Problem_OP_iR` class, which inherits from `pymoo.Problem` and encapsulates the simulation-based fitness evaluation. For each candidate design the class: (i) verifies feasibility through peak-gain and f_{sw} constraints; (ii) simulates all high-power operating points in parallel via `test_edge_cases`; and (iii) computes a scalar fitness score defined as the power mean of the RMS resonant current across all operating points, subject to true ZVS constraints.

- `main_ga.py` — configures and launches the genetic algorithm using `pymoo`'s GA solver with simulated binary crossover and polynomial mutation. The script defines the population size, number of generations, operating-point sets, and frequency bounds, then executes the optimization loop.

Furthermore, five example scripts are provided to demonstrate the feasibility of the framework and showcase its use:

- `Example.py` — minimal end-to-end verification script. Builds the LLC circuit for a fixed set of parameters, runs a single transient simulation for a specified number of switching cycles, extracts the steady-state average output voltage, and plots the full transient waveform. Intended to confirm that the PySpice/ngspice environment is correctly installed.
- `Example_Simulate_Until_SS.py` — demonstrates the `simulate_until_steady_state` function. Runs the block-by-block simulation until convergence is detected and then produces detailed steady-state waveform plots, including output voltage, resonant current, primary-side voltage, and secondary diode currents.
- `Example_Find_Vpeak_ss.py` — illustrates the use of `find_maximum_output_voltage_ss` to locate the frequency $f_{sw@Vpk}$ that results in maximum output voltage for a given converter design.
- `Example_find_Vo_target_single.py` — provides a convergence visualization for `find_vo_target` on a single operating point. Tracks every Brent iteration and generates a three-panel convergence plot showing the evaluated switching frequency, the simulated output voltage, and the absolute error as functions of the iteration count.
- `Example_find_Vo_target.py` — evaluates multiple operating points in parallel using `test_edge_cases`. For each entry in the user-defined operating-point list, the script finds the required switching frequency, collects all electrical metrics, and produces a consolidated comparison table.

III. SIMULATION-BASED ANALYSIS

Characterizing an LLC resonant converter inherently requires identifying its peak voltage (V_{pk}) and the f_{sw} required to synthesize a target V_o ($V_{o,target}$). While analytical methods rely on closed-form equations or numerical root-finding solvers, a simulation-based analysis approach requires dedicated algorithms to extract these metrics directly from the transient simulation. Therefore, in this section the analysis routines implemented in the `llc_sim` module are described. Initially, for the sake of clarity, the circuit model employed in the simulation-based analysis is first described, followed by the transient simulation procedure. Then, the periodic steady-state detection algorithm is presented, which is used as a subroutine in both `find_vo_target` and `find_maximum_output_voltage_ss`.

A. Circuit Model Description

The simulation model implements a half-bridge LLC resonant converter using a full-wave rectifier. The following subsections describe each stage of the model.

1) *Excitation Source*: The half-bridge switching action is represented by an ideal pulse-voltage source connected between nodes `vab` and `gnd` (ground). The source produces a unipolar square wave with amplitude V_{bus} , a period of $T_{sw} = 1/f_{sw}$, a duty cycle of approximately 50%, and the transition edges are set to 20ns. This abstraction eliminates the switching device models from the simulation, reducing solve time while preserving the steady-state resonant-tank waveforms with high fidelity.

2) *Resonant Tank and Transformer Subcircuit*: The LLC resonant tank consists of three passive elements, connected in the following way:

- C_s — series resonant capacitor, placed between nodes `vab` and `cs`;
- L_s — series resonant inductor, placed between nodes `cs` and `pri`;
- L_m — magnetizing inductance, connected from node `pri` to `gnd`, in parallel with the primary winding of the transformer.

The transformer is represented by the `TrxSubCir_CT` subcircuit, which implements a center-tapped (CT) model. The primary port is connected to nodes `pri` and `gnd`; the two secondary taps are connected to nodes `secn1` and `secn2`, both referenced to `gnd`. The subcircuit is parameterized by the turns ratio n (primary to each secondary), the primary inductance L_1 , and the winding copper resistances R_{c1} (primary) and R_{c2} (secondary). Default values employed are listed in Table I.

TABLE I
TRANSFORMER SUBCIRCUIT PARAMETERS AND DEFAULT VALUES.

Parameter	Default value	Description
n	—	Turns ratio, primary to each secondary. Defined by the user.
L_1	1000 H	Primary magnetizing inductance used internally by the subcircuit. Set large to approximate an ideal transformer.
R_{c1}	0.01 Ω	Primary winding copper resistance.
R_{c2}	0.01 Ω	Secondary winding copper resistance (each half).

3) *Output Rectifier, filter and Snubber Network*: Full-wave rectification is achieved through a centre-tap configuration using two diodes:

- D_1 : anode at `secn1`, cathode at `vo`;
- D_2 : anode at `secn2`, cathode at `vo`.

An RC snubber is connected in parallel with each diode to enhance numerical convergence. Each snubber consists of a 100 Ω resistor in series with a 470 pF capacitor.

Both rectifier diodes (D_1 and D_2) share the same SPICE level-1 model, `MyDiode`, whose parameters are given in Table II. The model omits junction capacitance and reverse-recovery charge, which is consistent with the ideal-source abstraction adopted for the primary side.

TABLE II
MYDIODE SPICE MODEL PARAMETERS.

Parameter	Value	Description
IS	10^{-7} A	Saturation (reverse leakage) current.
N	1.0	Ideality factor. A value of 1 corresponds to ideal diffusion-dominated conduction.
RS	0.1Ω	Ohmic series resistance, representing contact and bulk resistance of the diode.

Finally, the output stage consists of a filter capacitor C_o in parallel with the load resistance R_{load} , both connected between node v_o and gnd .

4) *Current Probes*: Zero-voltage current probes are inserted on the positive terminals of the following elements to allow post-processing of waveforms and loss metrics: L_s , L_m , R_{load} , D_1 , D_2 , and C_o . These probes add no electrical loading to the circuit.

B. Transient simulation

To illustrate how an LLC converter transient simulation is performed using ngspice and PySpice, we developed the example script `Example.py`. The script executes a single fixed-length (T_{end}) transient simulation for a given set of converter parameters, extracts the average output voltage ($V_{o,avg}$), and plots the output voltage waveform over the entire simulated interval. The Algorithm 1 presents the pseudocode of the simulation procedure.

Algorithm 1: Basic LLC Transient Simulation

Input: V_{bus} , n , C_s , L_s , L_m , C_o , R_{load} , f_{sw} ; $N_{sim,cyc}$, N_{PTsw} ;
Output: Average output voltage $V_{o,avg}$; waveform $v_o(t)$

- 1 $T_{sw} \leftarrow 1/f_{sw}$;
- 2 $\Delta t \leftarrow T_{sw}/N_{PTsw}$;
- 3 $T_{end} \leftarrow N_{sim,cyc} \cdot T_{sw}$;
- 4 $\mathcal{C} \leftarrow \text{BUILDCIRCUIT}(\text{Input parameters})$;
- 5 $\mathcal{A} \leftarrow \text{TRANSIENT}(\mathcal{C}, f_{sw}, \Delta t, T_{end})$;
- 6 $\mathcal{R} \leftarrow \text{ANALYZE}(\mathcal{A}, f_{sw}, \Delta t, N_{sim,cyc})$;
- 7 $V_{o,avg} \leftarrow \mathcal{R}.\text{voutAVG}$;
- 8 PLOT $v_o(t)$ over $[0, T_{end}]$;
- 9 **return** ($V_{o,avg}$, $v_o(t)$);

Input Parameters: The converter is fully described by a Python dictionary `params`, which collects all electrical quantities required to assemble the circuit netlist. Table III lists the parameters used in the script together with their values and physical meaning.

Simulator Configuration: Before the simulation is launched, two derived quantities are computed from the parameters above. The switching period is $T_{sw} = 1/f_{sw}$, and the simulation time step (Δt) is

$$\Delta t = \frac{T_{sw}}{N_{PTsw}} = \frac{1}{f_{sw} N_{PTsw}}. \quad (1)$$

TABLE III
CONVERTER PARAMETERS USED IN `EXAMPLE.PY`.

Key	Value	Description
Vbus	420 V	DC bus (input) voltage.
n	3.4	Transformer turns ratio (primary to each secondary half).
Cs	4.4 nF	Series resonant capacitor.
Ls	250 μ H	Series resonant inductor.
Lm	893 μ H	Transformer magnetising inductance.
Co	20 μ F	Output filter capacitor.
Rload	100 Ω	Equivalent DC load resistance.
fsw	120 kHz	Switching frequency.
NPTsw	250	Number of time steps per switching period; controls time resolution.
Nsimcyc	500	Number of switching cycles to simulate.

The total simulation time is $T_{end} = N_{sim,cyc} \cdot T_{sw}$, where $N_{sim,cyc} = 500$ switching cycles is the number of cycles requested.

Transient Simulation: Once the parameters are set, the simulation proceeds in three sequential steps. First, `build_llc_circuit` assembles the complete ngspice netlist from `params`, returning a `Circuit` object $\mathcal{C}$. Second, `Transient` configures the ngspice transient analysis with the time step Δt , the end time T_{end} , and the solver options listed in Table IV, and runs the simulation, returning the raw analysis object $\mathcal{A}$ that holds all node waveforms sampled at every Δt . Third, `analyze` post-processes $\mathcal{A}$ over the last 20 switching cycles, computing scalar performance metrics — including average of the output voltage $V_{o,avg}$, RMS and peak currents in the resonant tank, secondary diodes, and output capacitor. The script then prints $V_{o,avg}$ to the console and plots $v_o(t)$ over the full transient interval $[0, T_{end}]$, providing a direct visual indication of the voltage build-up from the initial condition ($v_o = 0$) to the steady state.

TABLE IV
DEFAULT NGSPICE SOLVER OPTIONS (`DEFAULT_OPTS`).

Parameter	Default value	Description
method	gear	Numerical integration method.
maxord	3	Maximum order of the Gear method.
reltol	10^{-3}	Relative convergence tolerance applied to all node voltages and branch currents.
abstol	10^{-6} A	Absolute current tolerance.
vntol	10^{-4} V	Absolute nodal voltage tolerance.
chgtol	10^{-12} C	Absolute charge tolerance.
cshunt	10^{-15} F	Shunt capacitance added to every node for numerical regularization. Prevents floating nodes from causing convergence failures.
rshunt	$10^9 \Omega$	Shunt resistance added to every node. Complements <code>cshunt</code> in stabilizing high-impedance nodes.
itl14	200	Maximum number of Newton–Raphson iterations allowed per time step during transient analysis.

C. Simulate until steady-state

Accurate analysis of switching converters requires that voltages and currents be obtained only after the system has reached periodic steady state (PSS). In practice, determining this point is challenging, since lightly damped systems can exhibit long transients and the settling time is rarely predictable. Traditional transient simulations, though straightforward, often result in either premature termination with nonconverged data or unnecessarily long runtimes due to redundant cycles. This inefficiency is a well-known limitation in conventional simulation-based analysis of power electronic circuits.

To overcome these issues, the proposal is to retain transient simulation but introduce an adaptive block-based detection mechanism. So, the simulation is divided into blocks, each comprising a fixed number of switching cycles. At the end of each block, the final states of the storage elements are carried forward as initial conditions, preserving continuity without reinitialization. After each block, convergence tests are applied, and the simulation is terminated immediately once PSS is confirmed.

This algorithm is implemented in `simulate_until_steady_state` that advances the transient simulation in successive blocks b of N_{cyc} switching cycles, carrying the final circuit state ($\mathbf{x}$) as initial conditions to the next block. Convergence is assessed at the end of each block by three independent criteria evaluated over a window of the last $N_{cyc,avg}$ cycles.

The Algorithm 2 presents the pseudocode of the proposed PSS detection procedure, and the main steps are detailed as follows:

Step 1 — Initialization. The block counter b , the stable-block counter s , and all previous-block references are set to zero or empty. The initial-condition map $\mathbf{x}$ is empty, so the first block starts from $v_o(0) = 0$.

Step 2 — Block simulation. A transient analysis runs for N_{cyc}/f_{sw} seconds. If the solver fails, the routine returns (None, None) immediately.

Step 3 — Block metrics. At the end of each block (b), three scalar metrics are computed: i) the average output voltage ($\bar{V}_o$) over the last cycles $N_{cyc,avg}$ of the block, given by (2); ii) the phase-corrected Poincaré sample $V_{\phi,b}$, defined as the interpolated value of v_o at the exact cycle boundary $t_b = b \cdot N_{cyc} \cdot T_{sw}$, given by (3); iii) the peak-to-peak ripple amplitude (ΔV_o) computed by (4), wherein W_b is the time window of the last $N_{cyc,avg}$ cycles of block b .

$$\bar{V}_{o,b} = \frac{1}{N_{cyc,avg} T_{sw}} \int_{t_b - N_{cyc,avg} T_{sw}}^{t_b} v_o(t) dt \quad (2)$$

$$V_{\phi,b} = v_o(b \cdot N_{cyc} \cdot T_{sw})|_{\text{interp}} \quad (3)$$

$$\Delta V_{o,b} = \max_{t \in W_b} \{v_o(t)\} - \min_{t \in W_b} \{v_o(t)\} \quad (4)$$

Step 4 — Criteria evaluation.

These metrics are compared against the preceding block. Once the relative errors of all criteria fall below predefined tolerances for a specified number of consecutive blocks, PSS is confirmed, and the solver is immediately interrupted.

Algorithm 2: Adaptive Block-Based PSS Detection

Input : $f_{sw}, NP_{Tsw}, N_{cyc}, N_{cyc,avg}, N_{cyc,max}, \varepsilon, \gamma, M$
Input : Circuit parameters $\theta = \{L_s, C_s, L_m, n, V_{in}, R_{load}\}$
Output : Block transient analysis $\mathcal{A}_b$

1 $b \leftarrow 0; s \leftarrow 0; \mathbf{x}_0 \leftarrow \mathbf{0}; \bar{V}_o^P \leftarrow 0; V_{\phi}^P \leftarrow 0; \Delta V_o^P \leftarrow 0$

2 **while** $b \cdot N_{cyc} < N_{cyc,max}$ **do**

3 $b \leftarrow b + 1$

4 $\mathcal{A}_b \leftarrow \text{TRANSIENT}(N_{cyc} \cdot T_{sw}, t_{step}, \mathbf{x}_{b-1})$

5 $\bar{V}_{o,b} \leftarrow (N_{cyc,avg} T_{sw})^{-1} \int_{W_b} v_o(t) dt$

6 $V_{\phi,b} \leftarrow \text{INTERP}(v_o, t_b)$

7 $\Delta V_{o,b} \leftarrow \max_{W_b} \{v_o\} - \min_{W_b} \{v_o\}$

8 **if** $\bar{V}_o^P \neq 0$ **then**

9 $e_1 \leftarrow |\bar{V}_{o,b} - \bar{V}_o^P| / |\bar{V}_o^P|$

10 $e_2 \leftarrow |V_{\phi,b} - V_{\phi}^P| / |V_{\phi}^P|$

11 $e_3 \leftarrow |\Delta V_{o,b} - \Delta V_o^P| / |\Delta V_o^P|$

12 **if** $e_1 < \varepsilon$ **and** $e_2 < \varepsilon$ **and** $e_3 < \gamma \varepsilon$ **then**

13 $s \leftarrow s + 1$

14 **else**

15 $s \leftarrow 0$

16 **end**

17 **if** $s \geq M$ **then**

18 **return** ($\mathcal{A}_b$)

19 **end**

20 **end**

21 $\mathbf{x}_b \leftarrow \{v_n(t_b)\}_{n \in \mathcal{N}} \cup \{i_L(t_b)\}_{L \in \mathcal{L}}$

22 $\bar{V}_o^P \leftarrow \bar{V}_{o,b}; V_{\phi}^P \leftarrow V_{\phi,b}; \Delta V_o^P \leftarrow \Delta V_{o,b}$

23 **end**

24 **return** ($\mathcal{A}_b$)

25 $W_b = [t_b - N_{cyc,avg} T_{sw}, t_b]$: time window of the last $N_{cyc,avg}$ cycles of block b , where $t_b = b \cdot N_{cyc} \cdot T_{sw}$;

27 $t_{step} = T_{sw} / NP_{Tsw}$: frequency-adaptive simulation time step;

28 Superscript p: denotes the stored value from the preceding block

29 ε : relative convergence tolerance;

30 $\gamma > 1$: relaxation factor for the ripple criterion;

31 M : consecutive stable blocks required to confirm PSS;

32 s : stable-block counter, reset upon any criterion failure.

Convergence criteria are evaluated relative to the previous block (skipped on the first block):

$$e_{\bar{V}} = \frac{|\bar{V}_{o,b} - \bar{V}_{o,b-1}|}{|\bar{V}_{o,b-1}|} < \varepsilon \quad (5)$$

$$e_{\phi} = \frac{|V_{\phi,b} - V_{\phi,b-1}|}{|V_{\phi,b-1}|} < \varepsilon \quad (6)$$

$$e_{\Delta V} = \frac{|\Delta V_{o,b} - \Delta V_{o,b-1}|}{|\Delta V_{o,b-1}|} < \gamma \varepsilon \quad (7)$$

Step 5 — Stable-block counter. If all three criteria pass simultaneously, s is incremented; otherwise, it is reset to zero. Steady state is declared when s consecutive blocks satisfy all criteria, and the routine returns the last block analysis $\mathcal{A}_b$.

Step 6 — State handoff. If convergence is not yet reached, the final node voltages (v_o, v_{Cs}) and inductor currents (i_{Ls}, i_{Lm}) are extracted from the simulation result and stored in **IC**. These are applied as initial conditions for the next block, ensuring state continuity and eliminating the transient that would otherwise arise from restarting the solver. The block metrics replace their previous-block counterparts and the loop repeats.

If the total cycle count reaches $N_{cyc,max}$ without converging, the last available analysis is returned, allowing the caller to

detect and handle the non-convergence case.

D. Find switching frequency

The function `find_vo_target` determines the switching frequency f_{sw} at which the LLC converter produces a required target output voltage $V_{o,target}$. The Algorithm 3 presents the pseudocode of the proposed procedure, and the main steps are detailed as follows:

Step 1 — Search bounds: Initially, the output voltage at series resonance is computed, as $V_o(f_o) = V_{bus}/(2n)$. Thus, if $V_{o,target} \geq V_o(f_o)$, and resonant tank provides a peak voltage higher than $V_{o,target}$, the search interval is set to $[f_{sw@Vpk}, f_o]$; otherwise the converter operates above f_o and the interval is $[f_o, f_{sw,max}]$.

Step 2 — Root-finding loop: Brent's bracketed root-finding method (`brentq`) is applied to the signed residual:

$$r(f_{sw}) = V_o(f_{sw}) - V_{o,target}, \quad (8)$$

where each evaluation of $V_o(f_{sw})$ calls `simulate_until_steady_state` and returns the average over the last $N_{cyc,avg}$ cycles. An early-exit mechanism interrupts the search as soon as $|r(f_{sw})| \leq \epsilon_{tol}$, avoiding unnecessary evaluations once the solution is already within tolerance.

Algorithm 3: Find f_{sw} required to achieve $V_{o,target}$

Input: V_{bus} , n , C_s , L_s , L_m , $f_{sw,max}$, $f_{sw@Vpk}$, $V_{o,target}$, ϵ

Output: Steady-state metrics $\mathcal{R}$; and $f_{sw,target}$

```

1  $f_o \leftarrow (2\pi\sqrt{C_s L_s})^{-1}$ 
2  $V_{o@fo} \leftarrow V_{bus}/(2n)$ 
3 if  $V_{o,target} \geq V_{o@fo}$  then
4    $(f_{low}, f_{high}) \leftarrow (f_{sw@Vpk}, f_o)$ 
5 else
6    $(f_{low}, f_{high}) \leftarrow (f_o, f_{sw,max})$ 
7 Function  $\text{Residual}(f_{sw})$ :
8    $V_{o,avg} \leftarrow \text{PSS detection at } f_{sw}$ ;
9   if  $|V_{o,avg} - V_{o,target}| \leq \epsilon$  then
10    Set  $f_{sw,target} \leftarrow f_{sw}$  and halt solver
11  return  $V_{o,avg} - V_{o,target}$ ;
12  $(r_l, r_h) \leftarrow (\text{Residual}(f_{low}), \text{Residual}(f_{high}))$ ;
13 if  $r_l \cdot r_h < 0$  then
14    $f_{sw,target} \leftarrow \text{BRENTQ}(\text{Residual}, f_{low}, f_{high})$ 
15 else
16    $f_{sw,target} \leftarrow \text{argmin}_{f \in [f_{low}, f_{high}]} |\text{Residual}(f)|$ 
17  $\mathcal{R} \leftarrow \text{ANALYZE}(f_{sw,target})$ ;
18 return  $(\mathcal{R}, f_{sw,target})$ ;
```

19 r_l : residual evaluated at the lower bracket endpoint f_{low} ;
20 r_h : residual evaluated at the upper bracket endpoint f_{high} ;
21 $\mathcal{R}$: complete set of steady-state operational metrics (RMS currents, peak voltage stresses, efficiency).;
22 $\text{ANALYZE}(\cdot)$: post-processing function that extracts steady-state metrics from the PSS simulation waveforms at a given f_{sw} .;
23 ϵ : convergence tolerance on $|V_{o,avg} - V_{target}|$.

Step 3 — Fallback strategy: If the residual has the same sign at both bounds, the algorithm falls back to `minimize_scalar` with Brent's bounded method applied to $|r(f_{sw})|$, returning the closest reachable frequency.

Step 4 — Final simulation: Once f_{sw} is determined, a final call to `simulate_until_steady_state` establishes the exact number of cycles N_{cyc} required to reach steady state at that frequency. The waveforms from this simulation are passed to `analyze()`, which returns the complete set of steady-state metrics.

E. Find peak voltage

During the analysis and design of an LLC resonant converter, it is essential to find the maximum output voltage that this converter can achieve. For the LLC resonant converter, the frequency search bounds to find the maximum output voltage are derived analytically from the tank components. The lower bound corresponds to the no-load resonant frequency f_1 given by (9), and the upper bound is set slightly above the series resonant frequency f_o (10) to ensure the peak of the voltage gain curve is captured. Regardless of the load, the peak gain is between these bounds.

$$f_{\min} = f_1 = \frac{1}{2\pi\sqrt{C_s(L_s + L_m)}} \quad (9)$$

$$f_{\max} = f_o + 1 \text{ kHz} = \frac{1}{2\pi\sqrt{C_s L_s}} + 1 \text{ kHz} \quad (10)$$

The function `find_maximum_output_voltage_ss` determines the switching frequency $f_{sw@Vpk}$ that maximizes the steady-state output voltage of the LLC converter over a prescribed search interval $[f_{\min}, f_{\max}]$. The Algorithm 4 presents the pseudocode of the proposed procedure, and the main steps are detailed as follows:

Step 1 — Initialization: The module-level simulation cache is cleared at the start of each call so that results from previous invocations do not contaminate the current search.

Step 2 — Scalar minimization: The problem is cast as a bounded scalar minimization:

$$f_{sw@Vpk} = \arg \min_{f_{sw} \in [f_{\min}, f_{\max}]} J(f_{sw}) = -\bar{V}_o(f_{sw}), \quad (11)$$

where the negative sign converts the maximization into a minimization. Brent's bounded method (`minimize_scalar`, `bounded`) is applied with absolute frequency tolerance $x_{atol} = 500 \text{ Hz}$ and a maximum of 20 iterations.

Step 3 — Cached objective evaluation: Each time the optimizer queries J at a candidate f_{sw} , the function `_get_vout_cached` first checks whether f_{sw} (rounded to 0.01 Hz) is already present in the cache. On a cache hit, the stored value is returned immediately at zero simulation cost. On a miss, `simulate_until_steady_state` is called to advance the circuit in blocks of N_{cyc} cycles until the three convergence criteria (mean, Poincaré, ripple) are satisfied or $N_{cyc,max}$ cycles are exhausted. The resulting average output voltage $\bar{V}_o$ is stored in the cache and returned as $J = -\bar{V}_o$.

Step 4 — Final analysis at $f_{sw@Vpk}$: Once the optimizer converges, a final call to `simulate_until_steady_state` with `save_all=True` records all node waveforms

Algorithm 4: Peak-Gain Tracking

Input : Circuit parameters $\theta = \{L_s, C_s, L_m, n, V_{in}, R_{load}\}$
Output : V_{pk} and $f_{sw,@Vpk}$

```

1  $f_m \leftarrow \frac{1}{2\pi\sqrt{C_s(L_s + L_m)}}$ 
2  $f_o \leftarrow \frac{1}{2\pi\sqrt{C_s L_s}}$ 
3  $\mathcal{C} \leftarrow \{\}$ 
4 Function  $\text{Objective}(f_{sw})$ :
5   if  $f_{sw} \in \mathcal{C}$  then
6      $V_o \leftarrow \mathcal{C}[f_{sw}]$ 
7   end
8   else
9     Simulate circuit to steady state (PSS Algorithm)
10     $\mathcal{C}[f_{sw}] \leftarrow V_o$ 
11  end
12  return  $-V_o$ 
13  $f_{sw,@Vpk} \leftarrow \arg \min_{f_{sw} \in [f_m, f_o]} \text{Objective}(f_{sw})$ 
14  $V_{pk} \leftarrow -\text{Objective}(f_{sw,@Vpk})$ 
15 return  $V_{pk}, f_{sw,@Vpk}$ 

```

16 $\mathcal{C}$: cache mapping each queried f_{sw} to its simulated V_o , preventing redundant SPICE runs when Brent's method re-evaluates a previously visited frequency;

at $f_{sw@Vpk}$. The complete set of steady-state metrics is then computed by `analyze()`.

Step 5 — Return values: The function returns the tuple $(f_{sw@Vpk}, V_{pk})$.